

FST Tanger

Pr. Ikram BENABDELOUAHAB

# **NLP PROJECTS**

---

*for master degree*

*2026*

## Instructions Générales pour tous les groupes

### 1. Interdiction d'utilisation directe des LLM

Il est **strictement interdit** d'utiliser une implémentation directe d'un LLM comme :

- Grok
- ChatGPT
- Claude

Cela inclut :

- copier/coller des réponses générées
- utiliser directement ces outils comme "boîte noire"
- générer des réponses sans pipeline RAG

Ce qui est attendu :

- utilisation d'un LLM **uniquement dans un pipeline RAG**
- contrôle et conception du prompt
- intégration avec un module de retrieval
- justification des résultats

L'objectif est d'évaluer :

- votre compréhension de l'architecture RAG
- votre capacité à construire un système complet
- et non l'utilisation brute d'un outil existant

### 2. CONTRAINTE IMPORTANTE (OBLIGATOIRE)

**Base de données personnelle uniquement**

Vous devez **obligatoirement construire votre propre base de données**.

**Il est strictement interdit d'utiliser :**

- datasets Kaggle existants
- datasets publics pré-étiquetés (phishing, spam, médical, juridique, etc.)
- bases de données prêtes à l'emploi issues de GitHub ou de publications

**Vous devez créer vous-même votre dataset à partir de :**

- génération manuelle de données adaptées à votre projet
- scénarios réalistes (ex : Smart City, cybersécurité, santé, droit, phishing...)
- variation des styles (langage formel, informel, technique, messages courts, etc.)
- création d'exemples positifs et négatifs selon le contexte du projet

Exemple (cas phishing) :

- emails suspects et légitimes
- messages frauduleux (banque, livraison, réseaux sociaux...)
- SMS courts et messages urgents

**Le dataset doit être :**

- structuré (**CSV / JSON**)
- annoté (labels adaptés au projet)
- suffisamment diversifié
- conçu pour permettre un **retrieval efficace dans un pipeline RAG**

**Enrichissement obligatoire**

En plus de la création manuelle, vous devez intégrer :

**une génération automatique de données synthétiques** pour enrichir votre dataset (ex : génération de nouveaux scénarios, variations de messages, cas supplémentaires)

### 3. Objectifs pédagogiques globaux

Ces contraintes visent à :

- développer vos compétences en **construction de datasets**
- comprendre le fonctionnement réel du **RAG (retrieval + génération)**
- éviter la dépendance aux outils automatisés
- encourager la créativité et la réflexion critique

## Projet de Fin de Module

### Sujet 1 : Génération de règles YARA

Dans ce projet, vous allez concevoir et développer un système intelligent basé sur le NLP (Natural Language Processing) et le Retrieval-Augmented Generation (RAG) permettant de générer automatiquement des règles YARA à partir de descriptions en langage naturel.

Les règles YARA sont largement utilisées en cybersécurité pour la détection et l'identification de malwares. Leur rédaction est souvent manuelle, complexe et nécessite une expertise avancée.

L'objectif de ce projet est d'explorer comment les techniques modernes de LLM (Large Language Models) combinées à une approche RAG peuvent assister ou automatiser cette tâche.

#### Travail demandé

Vous devez développer un système capable de :

##### 1. Compréhension en langage naturel

- Analyser une requête utilisateur décrivant un comportement malveillant
- Exemple :

*“Détecter un ransomware qui chiffre les fichiers et utilise AES”*

##### 2. Module de récupération (Retrieval)

- Construire une base de connaissances contenant :
  - règles YARA existantes
  - descriptions de malwares
  - signatures connues
- Implémenter un moteur de recherche sémantique (FAISS, BM25 ou embeddings)

##### 3. Module de génération (LLM)

- Utiliser un modèle de langage (GPT, Qwen, LLaMA, Mistral, etc.)
- Benchmarking des modèles via une comparaison des métriques d'évaluation
- Générer une règle YARA en se basant sur :
  - la requête utilisateur
  - les documents récupérés

##### 4. Pipeline RAG complet

Le système doit suivre l'architecture suivante :

1. Entrée utilisateur (texte naturel)
2. Vectorisation de la requête
3. Récupération des documents pertinents (Top-k)
4. Enrichissement du prompt
5. Génération de la règle YARA

De plus :

- Évaluation de la qualité des règles YARA générées et clustering
- Explication automatique des règles générées

##### 4. Livrables

- Code source du système RAG
- Interface Web sous forme de dashboard, avec possibilité d'ajouter un nouveau PDF qui servira comme une nouvelle base de connaissance
- Dataset utilisé (ou construit)
- Rapport technique expliquant :
  - l'architecture
  - les choix techniques
  - les tests réalisés

##### 5. Critères d'évaluation

- Qualité de la récupération (retrieval)
- Pertinence des règles YARA générées
- Intégration correcte du pipeline RAG
- Clarté du code et de la documentation

## Projet de Fin de Module

### Sujet 2 : Génération de règles SNORT

L'objectif de ce projet est de développer un système intelligent basé sur le NLP et le Retrieval-Augmented Generation (RAG) capable de générer automatiquement des règles Snort à partir de descriptions en langage naturel de comportements réseau suspects.

Snort est un système de détection d'intrusion (IDS) utilisé pour analyser le trafic réseau et détecter des activités malveillantes. La création de règles Snort est une tâche complexe nécessitant une bonne connaissance des protocoles réseau et des signatures d'attaques.

Ce projet vise à exploiter les LLM + RAG pour assister ou automatiser cette tâche.

#### Travail demandé

##### 1. Compréhension en langage naturel

- Analyser une description utilisateur d'une attaque réseau
- Exemple :  
"Détecter une tentative de scan de ports sur un serveur web"

##### 2. Module de récupération (Retrieval)

Construire une base de connaissances contenant :

- règles Snort existantes
- descriptions d'attaques réseau (DoS, scan, injection, etc.)
- logs réseau ou signatures d'intrusion

Implémenter un moteur de recherche sémantique basé sur :

- embeddings (Sentence-BERT, etc.) et BM25
- Effectuer un benchmarking des embeddings models

##### 3. Module de génération (LLM)

Utiliser un modèle de langage (GPT, Nemotron, Mistral, LLaMA, etc.) pour :

- générer une règle Snort valide
- en se basant sur la requête + les documents récupérés
- Benchmarking des LLMs via métriques d'évaluation

##### 4. Pipeline RAG complet

Le système doit suivre le processus suivant :

1. Entrée utilisateur (description d'attaque)
2. Encodage de la requête
3. Récupération des règles Snort similaires (Top-k)
4. Construction d'un prompt enrichi
5. Génération de la règle Snort

De plus :

- Détection automatique de type d'attaque (classification) et famille d'attaque (clustering)
- Explication des règles Snort générées
- Optimisation des faux positifs
- Intégration avec logs réseau réels

#### Livrables

- Code source du système RAG
- Base de données de règles Snort via une dataset construit par vous-même
- Interface Web sous forme de dashboard, avec possibilité d'ajouter un nouveau PDF qui servira comme une nouvelle base de connaissance
- Rapport technique incluant : architecture du système, choix des modèles, exemples de génération, limites et améliorations possibles

Critères d'évaluation

- Pertinence des règles Snort générées
- Qualité du module de récupération
- Cohérence du pipeline RAG
- Qualité du code et documentation

## Projet de Fin de Module

### Sujet 3 : Détection et Analyse du phishing

L'objectif de ce projet est de développer un système intelligent basé sur le NLP et le Retrieval-Augmented Generation (RAG) permettant d'analyser des messages suspects (emails, SMS, URLs) afin de détecter des tentatives de phishing, tout en fournissant une explication détaillée.

Le phishing est une attaque de cybersécurité reposant sur la manipulation des utilisateurs (emails frauduleux, faux sites, messages urgents, etc.).

Dans ce projet, vous devez exploiter les techniques de LLM + RAG pour analyser ces contenus et assister la prise de décision.

Travail demandé

#### 1. Construction de la base de données

Créer une base de données personnelle contenant :

- emails légitimes
- emails de phishing
- SMS frauduleux
- messages réseaux sociaux suspects
- URLs et scénarios associés

Chaque entrée doit inclure :

- texte du message
- label (phishing / non phishing)
- type d'attaque (optionnel mais recommandé)
- niveau de risque (optionnel)

#### 2. Compréhension en langage naturel

Le système doit analyser un message fourni par l'utilisateur, par exemple :

"Votre compte a été suspendu, cliquez ici immédiatement pour le réactiver"

- classification multi-niveaux (faible / moyen / critique)
- support multilingue (FR / EN / AR)

#### 3. Module de récupération (Retrieval)

Construire un moteur de recherche sémantique basé uniquement sur votre base de données personnelle : embeddings (Sentence-BERT, MiniLM, etc.) et BM25

Le système doit récupérer les cas similaires dans votre dataset.

#### 4. Module de génération (LLM)

Utiliser un modèle de langage (GPT, Mistral, LLaMA, Qwen, etc.) pour :

- classer le message (phishing / non phishing)
- expliquer la décision
- citer les éléments suspects (urgence, lien, usurpation, etc.)
- Benchmarking des LLMs via métriques d'évaluation

#### 5. Pipeline RAG complet

Le système doit suivre le pipeline suivant :

1. Entrée utilisateur (message suspect)
2. Encodage du message
3. Recherche dans la base de données personnelle uniquement
4. Récupération des exemples similaires (Top-k)
5. Construction du prompt enrichi
6. Génération :
  - classification
  - explication
  - recommandations

Fonctionnalités clés :

- analyse d'URLs suspectes intégrée
- tableau de bord de visualisation des attaques

**6. Livrables attendus**

- Code source complet du système RAG
- Base de données personnelle (obligatoire)
- Description de la méthodologie de construction du dataset
- Interface Web sous forme de dashboard, avec possibilité d'ajouter un nouveau PDF qui servira comme une nouvelle base de connaissance
- Rapport technique incluant :
  - architecture du système
  - stratégie de génération du dataset
  - exemples de tests
  - analyse des performances

**7. Critères d'évaluation**

- qualité et originalité de la base de données créée
- pertinence du retrieval sur dataset personnel
- qualité des explications générées
- cohérence du pipeline RAG
- qualité du code et de la documentation

## Projet de Fin de Module

### Sujet 4 : Analyse des lois et de la justice au Maroc

Ce projet a pour objectif de développer un système intelligent basé sur le **NLP (Natural Language Processing)** et le **Retrieval-Augmented Generation (RAG)** permettant de répondre à des questions juridiques relatives au droit marocain, en s'appuyant sur une base de connaissances construite par les étudiants.

Le système devra fournir des réponses contextualisées concernant différents domaines juridiques au Maroc.

#### Domaines juridiques concernés

Le système doit couvrir plusieurs branches du droit marocain, clairement modifiable via l'interface web, notamment :

- Code de la route marocain
- Droit pénal (القانون الجنائي)
- Droit de la famille (مدونة الأسرة)
- Système judiciaire marocain (المحاكم: الابتدائية، الاستئناف، النقض)
- Droit civil et commercial
- Infractions et sanctions

#### Travail demandé

##### 1. Construction de la base de données (en arabe)

Créer une base de connaissances contenant :

- مخالفات قانون السير المغربي
- الجرائم والعقوبات في القانون الجنائي
- قضايا الأسرة (الطلاق، النفقة، الحضانة)
- إجراءات المحاكم المغربية
- حالات قانونية واقعية أو محاكاة

Chaque entrée doit inclure :

- النص القانوني (باللغة العربية)
- شرح مبسط (باللغة العربية)
- نوع القانون
- مثال تطبيقي

##### 2. Compréhension des requêtes en langage naturel

Le système doit traiter des questions juridiques en arabe, par exemple :

- "ما هي عقوبة تجاوز السرعة في الطريق السيار؟"
- "كيف يتم رفع دعوى الطلاق في المغرب؟"
- "ما هي إجراءات المحكمة الابتدائية؟"

##### 3. Module de récupération (Retrieval)

Mettre en place un moteur de recherche sémantique basé **uniquement sur la base de données arabe créée** :

- embeddings (Sentence-BERT, multilingual models, etc.) et BM25

Le système doit récupérer les textes juridiques les plus pertinents.

##### 4. Module de génération (LLM)

Utiliser un modèle de langage (GPT, Mistral, LLaMA, Qwen, etc.) pour :

- analyser la question juridique
- générer une réponse en arabe
- expliquer la règle juridique applicable
- fournir un raisonnement simple
- Benchmarking des LLMs via métriques d'évaluation

## 5. Pipeline RAG complet

Le système doit suivre les étapes suivantes :

1. Entrée utilisateur (question en arabe)
2. Vectorisation de la requête
3. Recherche dans la base de données juridique arabe
4. Récupération des documents pertinents (Top-k)
5. Construction du prompt en arabe
6. Génération de la réponse finale en arabe

## 6. Fonctionnalités clés

- classification automatique des types de litiges
- génération de scénarios juridiques marocains
- explication simplifiée des articles de loi
- support bilingue (arabe / français)
- interface utilisateur interactive

## 7. Livrables

- Code source du système RAG
- Base de données juridique en arabe
- Description de la méthodologie de création du corpus
- Interface Web sous forme de dashboard, avec possibilité d'ajouter un nouveau PDF qui servira comme une nouvelle base de connaissance. Choisir entre les bases de connaissances disponibles.
- Rapport technique incluant :
  - architecture du système
  - choix techniques
  - exemples de requêtes et réponses en arabe
  - limites et améliorations

## Critères d'évaluation

- qualité et originalité de la base de données en arabe
- pertinence des réponses juridiques
- efficacité du retrieval
- cohérence du pipeline RAG
- qualité du code et de la documentation



## Projet de Fin de Module

### Sujet 5 : Assistant Médical spécialiste en oncologie (références marocaines)

Ce projet vise à développer un assistant intelligent basé sur le NLP (Natural Language Processing) et le Retrieval-Augmented Generation (RAG) capable d'analyser des questions en oncologie et de fournir des réponses pédagogiques, structurées et contextualisées.

Le système devra s'appuyer sur des protocoles oncologiques reconnus au Maroc, tout en proposant une reformulation accessible aux utilisateurs (patients ou non spécialistes).

Les protocoles oncologiques contiennent des informations complexes sur :

- diagnostic des cancers
- stratégies thérapeutiques
- protocoles de traitement
- suivi des patients

Ces documents sont souvent difficiles à comprendre pour un non-spécialiste. L'objectif est donc de concevoir un système capable de :

- simplifier ces informations
- les structurer
- les rendre accessibles via une interface intelligente
- possibilité d'analyser et de proposer des traitement hybride à la base du guide des protocoles marocains

#### CONTRAINTE IMPORTANTE (OBLIGATOIRE)

##### Base de données personnelle à partir de références marocaines

Vous devez **obligatoirement construire votre propre base de données**, en vous appuyant sur des documents officiels marocains, notamment :

- AMFROM
- Guide des protocoles thérapeutiques en oncologie (édition récente)
- Source des données :  
<https://amfrom.ma/assets/pdfs/Guide%20des%20protocoles%202024.pdf>
- Ajouter d'autres du **Maroc**

#### Interdictions strictes

- copier directement le contenu du guide PDF
- utiliser le document comme dataset brut
- faire du copier-coller des protocoles médicaux
- importer automatiquement le PDF dans un système RAG

#### Travail attendu

Vous devez :

- lire et comprendre les protocoles
- reformuler les informations avec vos propres mots, bref et technique, sans une perte de données
- simplifier le contenu pour le rendre pédagogique
- structurer les données pour le retrieval

**Travail demandé****1. Construction de la base de données oncologique**

Créer une base de données contenant :

- types de cancers (sein, poumon, colorectal, etc.)
- étapes de diagnostic
- protocoles de traitement (simplifiés)
- effets secondaires
- parcours de soins
- scénarios patients fictifs

**Format des données**

Chaque entrée doit inclure :

- contenu reformulé
- catégorie (diagnostic / traitement / suivi)
- mots-clés
- scénario ou cas d'usage
- protocole : traitement dosage

En plus des données extraites et reformulées :

- ✓ générer des données synthétiques (exemples, scénarios patients, variations)
- ✓ diversifier les cas (stades, profils patients, situations cliniques de simples à complexes, cas des métastases divers et maladies chroniques)

**2. Compréhension NLP**

Le système doit répondre à des questions générales telles que :

- “Quel est le protocole de traitement du cancer du sein ?”
- “Quels sont les effets secondaires de la chimiothérapie ?”
- “Comment se déroule le diagnostic d'un cancer ?”

En plus de questions spécifiques pour assister rapidement un oncologue :

- Symptômes + données cliniques => protocole ou protocoles depuis le guide avec référence (ex. page et nom du référence ou encore année/version)

**3. Module de retrieval**

Mettre en place : embeddings (Sentence-BERT ou équivalent) et BM25

Recherche uniquement dans votre base de données construite

**4. Module de génération (LLM)**

Le système doit :

- générer des réponses claires et pédagogiques
- expliquer les protocoles de manière simplifiée
- structurer les informations (étapes, traitements, etc.)
- éviter tout diagnostic médical direct
- Un Benchmarking d'au moins 3 LLM est obligatoire avec comparaison et analyse des métriques d'évaluation

**5. Pipeline RAG**

1. Question utilisateur
2. Encodage
3. Recherche dans la base
4. Récupération Top-k
5. Prompt enrichi (Comparaison et analyse de plusieurs Prompt)

## 6. Génération

## 8. Livrables attendus

- Code source complet
- Base de données personnelle
- Description de la méthode d'extraction et reformulation
- Interface Web sous forme de dashboard, avec possibilité d'ajouter un nouveau PDF qui servira comme une nouvelle base de connaissance
- Rapport technique incluant : tout les détails techniques

## 9. Critères d'évaluation

- qualité de la reformulation des protocoles
- pertinence médicale (niveau pédagogique)
- qualité du retrieval
- cohérence du pipeline RAG
- qualité du code

## 10. Fonctionnalités clés

- classification des cancers
- explication des protocoles étape par étape (à la demande de l'utilisateur)
- visualisation du parcours patient exemplaire
- assistant conversationnel interactif en se basant principalement sur le guide de référence fourni